Methods — Canonical

Pre-processing

Smoothing/downsampling
Detrending
ROI extraction
Motion correction
Creating artificial trials
Spike detection/sorting
Calcium → spikes
Estimating firing rates from spikes
Pose/keypoint tracking
Principal components analysis

Classical signal processing

Covariance/correlation functions
Power spectra
Convolution theorem
Wiener-Khinchin theorem
Coherence & cross-spectral density
Impulse response
Linear filters
MA/AR/ARMA models

Canonical neural data analyses

Raster plots
Interspike interval distributions
Peristimulus time histogram (PSTH)
Tuning curves
Linear-nonlinear-Poisson (LNP)
Generalized linear model (GLM)
Spike-triggered average

Common pre-processing steps for neural and behavioral data

Smoothing/downsampling
Detrending
ROI extraction
Motion correction
Creating artificial trials

Spike detection/sorting
Calcium → spikes
Estimating firing rates from spikes
Pose/keypoint tracking
Principal components analysis

Major challenge:

Many small decisions to make (e.g. choosing parameters)
Impractical to keep everything as a free parameter

Helpful rules of thumb:

Follow standard practice/expectations in the field (if any)
Be cognizant of downstream effects of pre-processing choices
Keep specific scientific questions in mind when deciding on pre-processing

Pre-processing: Smoothing/downsampling

- Variability in time-series data is often “noise” that one would like to ignore in one’s analysis.

- Smoothing: replacing x(t) with weighted sum of x(t-T), x(t-T+1), …, x(t), …, x(t+T-1), x(t+T).
  - Common weightings:
    - Box-car/moving average: all timepoints within a specific window weighted equally
    - Exponential filter: weights exponentially decay going back into the past
  - To preserve causality, only weight timepoints <= t (common for exponential filter).

- Downsampling: creating smaller number of time bins than original signal
  - Each time bin can be (1) weighted average of other time bins, (2) single time bin from original signal

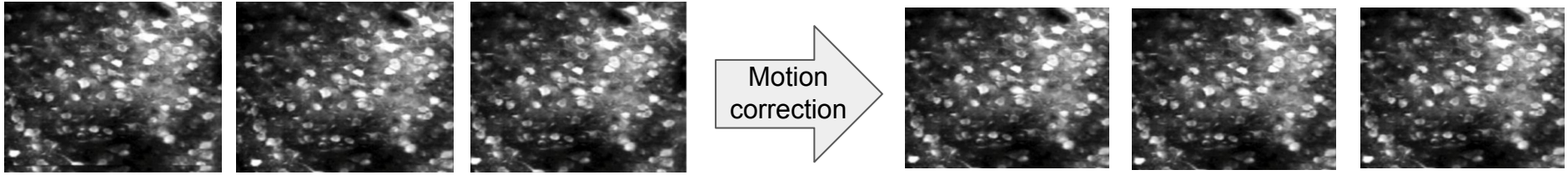
- Smoothing/downsampling throw away high-frequency information (like low-pass filter).

- Not always principled way to pick smoothing window size (too fine does nothing, too coarse throws out the signal with the noise). In general keep the scientific questions in mind to make such decisions.

Pre-processing: Detrending

- Meant to account for slow drift in signal due to experimental artifact (e.g. photobleaching of fluorophore).
- Simplest approach: fit $x(t) = at + b$, then remove this from $x(t)$.
- Be careful though, since drift can be nonlinear.
- Similar to high-pass filter.
- Dangerous if you believe slow drift is important (non-artifactual) part of signal, since this will be removed

Pre-processing: Motion correction



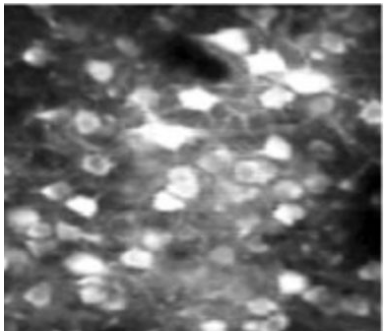
Adapted from Giovannucci et al, *eLife* 2019, CC-BY 4.0.

Variability in x, y, z planes

Z-plane problematic since can look like fluorescence increase

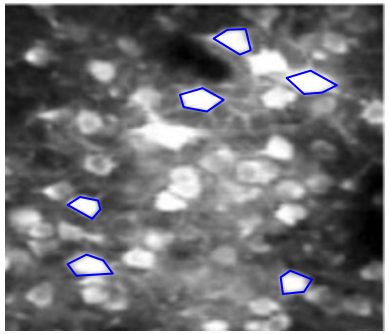
Software:
NoRMCorre
TMAC

Pre-processing: ROI extraction



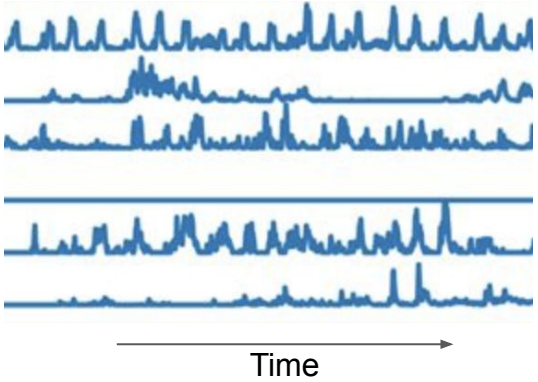
Adapted from Giovannucci et al, *eLife* 2019, CC-BY 4.0.

ROI extraction



ΔF/F

Adapted from Giovannucci et al, *eLife* 2019, CC-BY 4.0.



Algorithms:
ICA,
CNMF

Software packages:
Suite2p
CaImAn

Supervoxel

ΔF/F: Change in fluorescence (ΔF) relative to some baseline F

When “trials” present, F often defined as time-averaged fluorescence over some window before stimulus onset

Can also define F as time-average of whole time-series

Pre-processing: Creating artificial trials

Many naturalistic time-series are not segmented into trials.

However, they often include repeated “events” that can be thought of as trial onsets. E.g.:

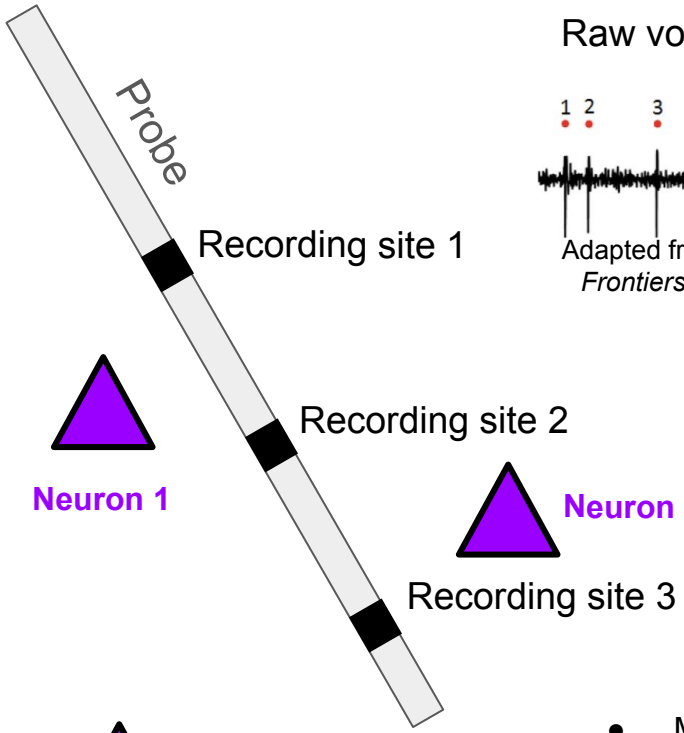
- Locomotion onsets when animals walk sporadically (Zolin et al 2021).
- Odor-plume crossings (van Breugel & Dickinson 2014).
- Onset of particular neural “hidden state” (Recanatesi et al 2022).

Can now perform statistics across trials, e.g. trial averages/variances.

Keep in mind:

- “Trials” might not be fully independent
- Trials can have variable length
- Trials can overlap if specific length is specified
    - E.g. define “trial” as 5-second window following locomotion onset, but then find time when many locomotion onsets occur within a 5-second window

Pre-processing: Spike detection and sorting



Raw voltages from electrode

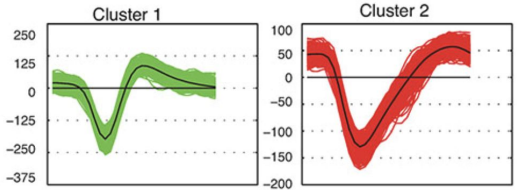
Adapted from Petrantonakis and Poirazi *Frontiers Neurosci* 2015, CC-BY 4.0.

Detect spikes

Spike waveforms from extracellular recordings are highly variable, but semi-consistent for same neuron

Adapted Petrantonakis and Poirazi *Frontiers Neurosci* 2015, CC-BY 4.0.

"Sort" spikes (assign neuron labels)



Neuron 1 Neuron 2

Adapted from Mokri et al *Frontiers Neuroinf.* 2017, CC-BY 4.0.

Challenges

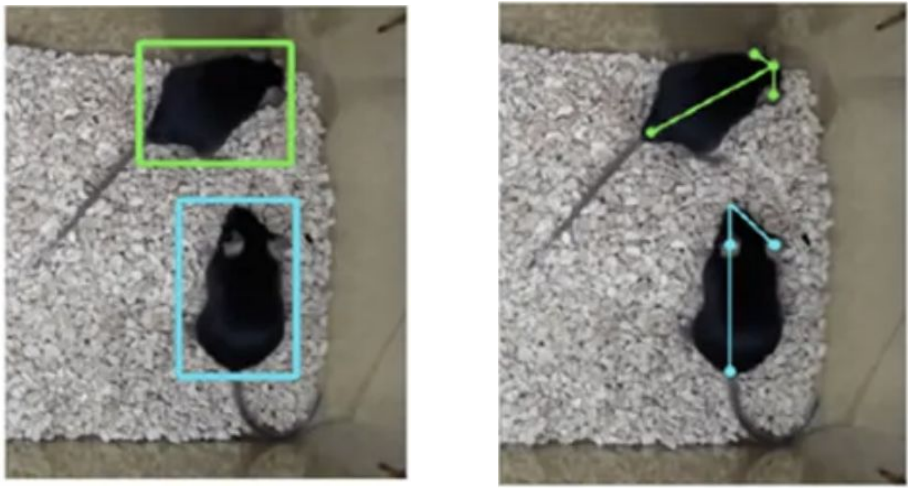
- Multiple neurons can have similar waveforms
- Waveforms drift over time
- Often some "experimenter intuition" required

Common software:

- Kilosort

Pre-processing: Pose/keypoint tracking

Keypoint estimation



Adapted from Chen et al, *Frontiers Behav Neurosci* 2023, CC-BY 4.0.

Software:

SLEAP
DeepLabCut
AlphaTracker
Anipose (3D)

Challenges:

Keypoints can be occluded

Usually needs some manual training data

Usually requires proofreading

Pre-processing: Estimating firing rates from spike trains

Average over trials (PSTH)

- Need multiple trials to average over
- Need to pick bin size

Average over time – binning

- Need to pick bin size

Average over time – kernel density estimator

- Convolve (usually squared exponential) “kernel” with spike train
- Need to pick kernel width

Infer firing rates as latent variables (advanced)

- Fit statistical model of form P(spikes(t)|r(t), other params)
- Given params can infer r(t), but might need to learn params too.
- Usually still need to specify timescale (or prior over timescale) of firing rate variability
    - Can sometimes optimize through cross validation

Classical signal processing methods

Correlation functions
Power spectra
Wiener-Khinchin theorem

Impulse response
Linear filters
MA/AR/ARMA

Advantages

Very well understood mathematically
Generic to time-series analysis, so can be applied widely

Challenges

Make sometimes strong assumptions, e.g. stationarity, linearity, additive noise…
Limited in what they can tell you (e.g. more descriptive than mechanistic)
Can require many samples for good performance
Neuroscience data often too far from assumptions for method to be relevant

Classical signal processing: Estimating correlation functions

Autocovariance & autocorrelation $C_X(\tau)$:

- Provides graphical summary/statistic of how one (scalar) signal X(t) changes over time
- Assumes stationarity of signal

Crosscovariance & cross-correlation $C_{XY}(\tau)$

- Provides graphical summary/statistic of how one signal X(t) temporally relates to another signal Y(t)
- Assumes stationarity of signal

Two-point covariance/correlation functions $C_{XY}(t_1, t_2)$:

- Covariance of $X(t_1)$, $Y(t_2)$
- No stationarity assumption required

Estimating of auto- and cross-covariance:

- Unbiased estimate: Directly estimate covariance of X(t), X(t+τ) pairs.
  - If no missing data can compute via convolution of X(t), X(t+τ), which can be done using FFT.
    - If using convolution, need to divide by correct number of points at each τ for unbiased est.
  - Gotcha: unbiased estimate can yield “invalid” function (e.g. $|C(0)| < |C(\tau)|$).
- Biased estimate: uses convolution but divides sum by same number of points at each τ
  - Sometimes default (e.g. in MATLAB)
  - Yields valid function, but will artificially suppress long-range correlations.

Covariance, correlation, covariance function, correlation function

Covariance: Cov(X, Y):

- Covariance between X and Y
- = E[(X-E[X])(Y-E[Y])] = E[XY] - E[X]E[Y]

Pearson correlation: R(X, Y):

- Covariance between X and Y, scaled to range between -1 and 1
- = E[(X-E[X])(Y-E[Y])]/(SD(X)SD(Y))

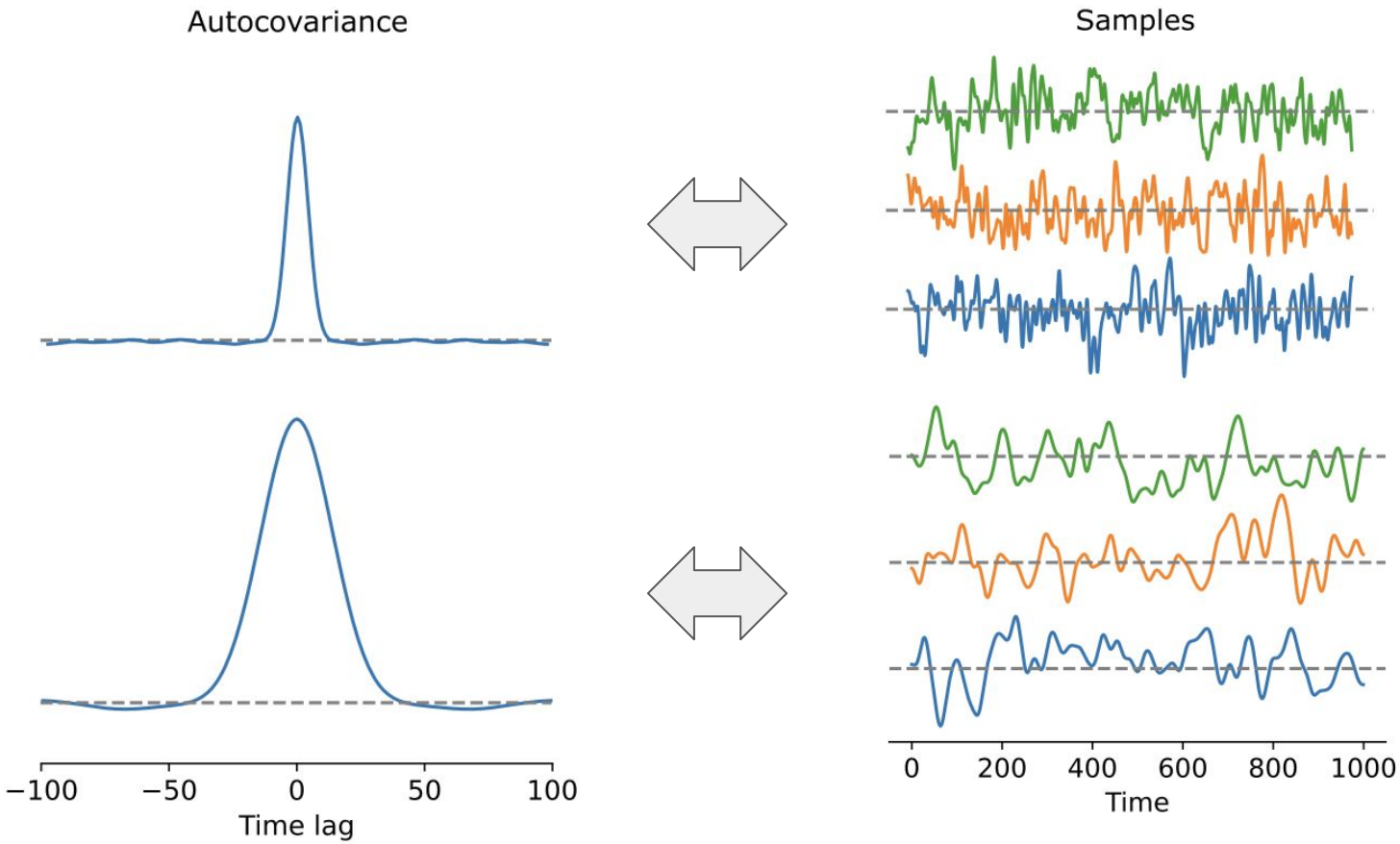
Cross-covariance Function: $Cov_{XY}(t_1, t_2)$

- = $Cov(X(t_1), Y(t_2))$
- = $Cov_{XY}(tau)$ = Cov(X(t), Y(t - tau)) for wide-sense stationary process
- Auto-covariance function: let X(t) = Y(t)

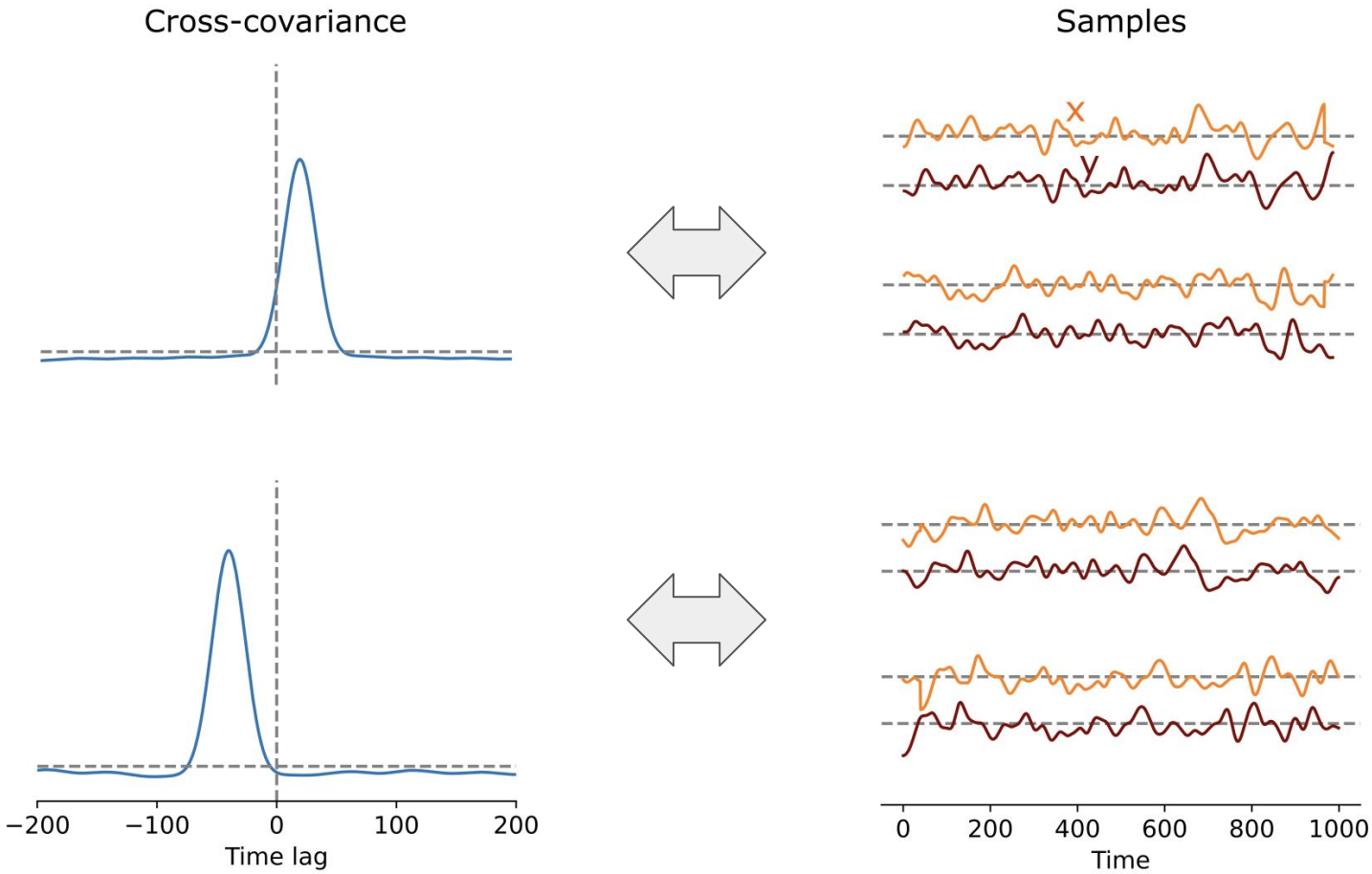
Cross-correlation Function: $R_{XY}(t_1, t_2)$

- $E[X(t_1)Y(t_2)]$ = Cov(X(t), Y(t-tau)) + E[X]E[Y]
- NOT constrained to be between -1 and 1
- Auto-correlation function: let X(t) = Y(t)

Classical signal processing: Autocovariance function examples



Classical signal processing: Cross-covariance function examples



Classical signal processing: auto/cross-covariance vs auto/cross-correlation

Correlation/covariance function often used interchangeably in informal discussion.

Auto/cross-correlation is NOT Pearson correlation.

- Pearson correlation is covariance divided by product of standard deviations.

Auto/cross-correlation function R is auto/cross-covariance function C plus product of means:

- $C_{XY}(\tau) = E_{XY}[\ (X(t) - E_X[X(t)])(Y(t+\tau) - E_Y[Y(t+\tau)])\ ] = E_{XY}[X(t)Y(t+\tau)] - E_X[X(t)]E_Y[Y(t)]$
- $R_{XY}(\tau) = E_{XY}[X(t)Y(t+\tau)] = C_{XY}(\tau) + E_X[X(t)]E_Y[Y(t)]$

Power spectral density

Captures how much “power” exists in different frequency bands in signal.

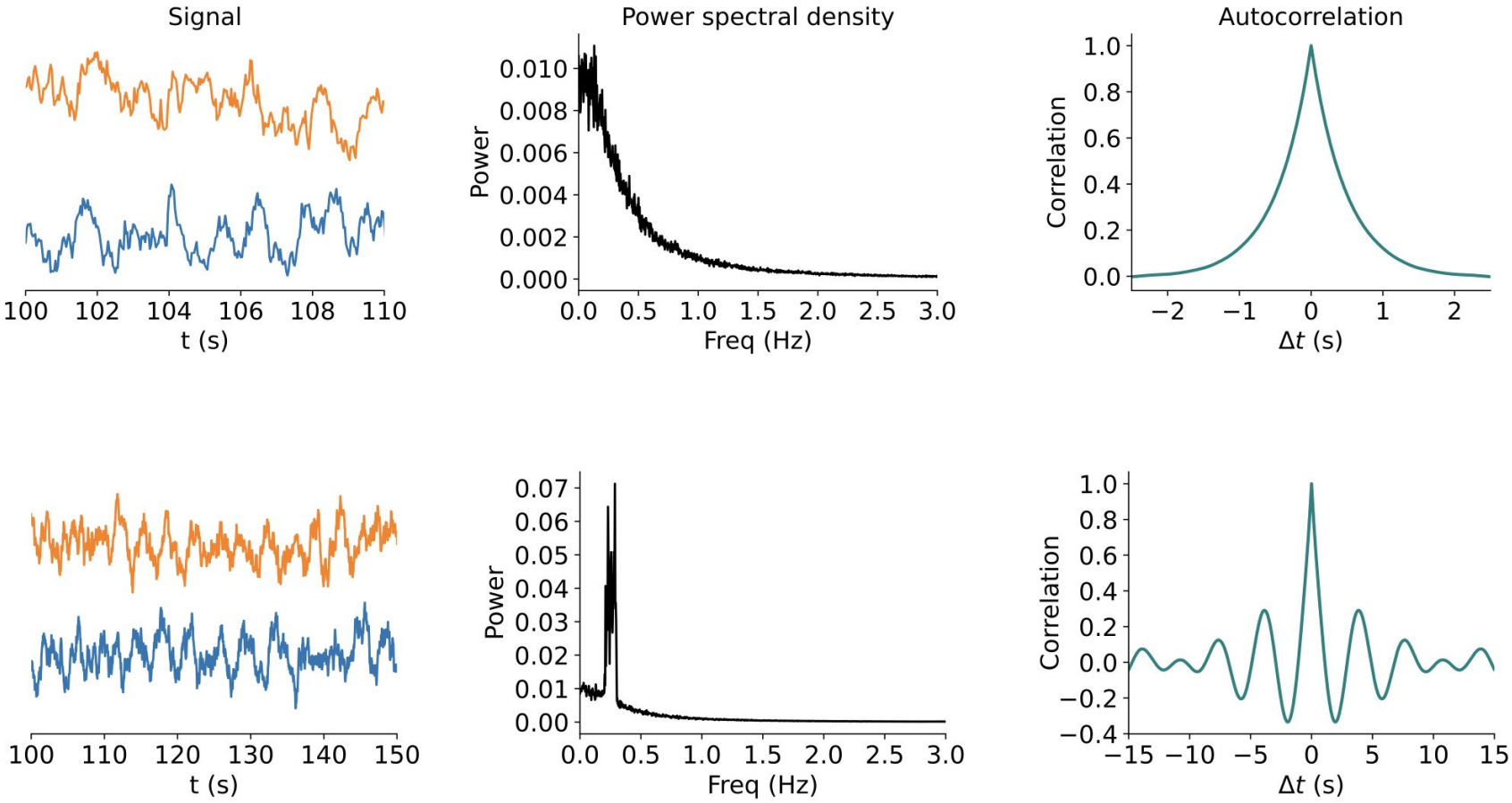
Useful when signal expected to have oscillations.

Strictly speaking, PSD is a statistic of an expectation value of random process.

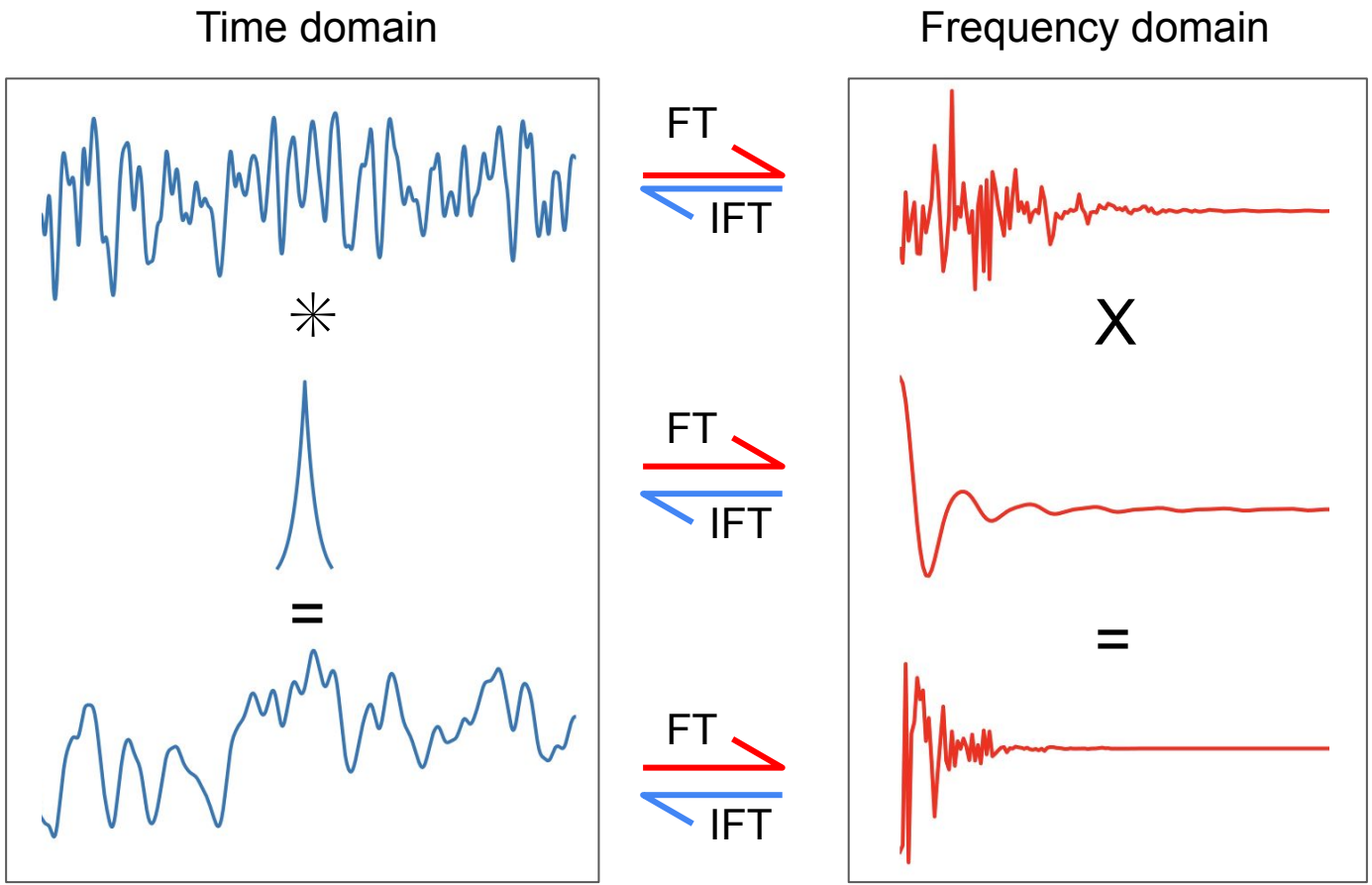
- PSD(f) = E[FFT[X(t)](f)*FFT[X(t)](f)]
- When we compute the FFT*FFT of a single signal instance X(t) we are estimating this quantity from a sample of the random process.

PSD units depends on signal units. If X(t) in Amp, then PSD in Amp^2/Hz.

Power spectral density examples



Convolution theorem: Convolution in time domain = multiplication in frequency domain



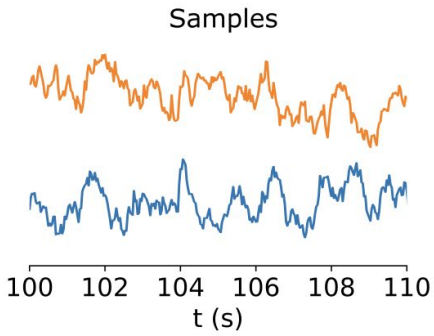
$$x(t) \circledast y(t) = \mathrm{IFT}[\ \mathrm{FT}[x(t)]\mathrm{FT}[y(t)]\ ]$$

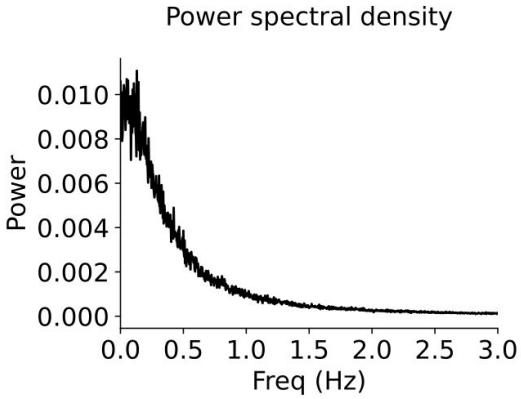
*More generally: convolution becomes multiplication after Fourier Transform

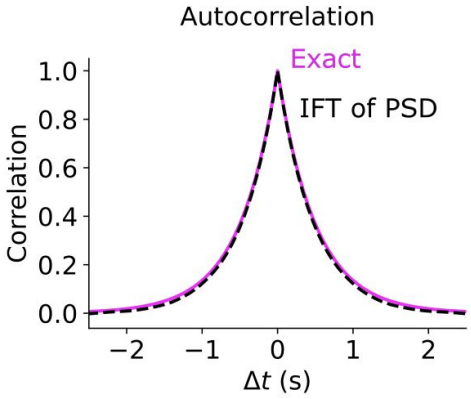
This is true for well-behaved x(t) and y(t), regardless of whether they were sampled from a random process

Wiener-Khinchin theorem

Power Spectral Density = Fourier Transform of Autocorrelation Function







Cross spectral density and coherence

Autocorrelation function ↔ Power spectral density

Cross-correlation function ↔ Cross spectral density

Consider one sample X(t), Y(t) from ensemble P[X(t), Y(t)]

- X(t) has some component at frequency f
- The strength of this component is random, since X(t) is random
- Y(t) has some component at frequency f
- The component of X(t) at f may covary with the component of Y(t) at f

Cross-spectral density CSD(f)

- Captures relationship between components at f in X(t) and Y(t)
- Can be complex valued
- = Fourier transform of cross-correlation function

Coherence $\text{Coherence}_{XY}(f)$

- Real-valued, normalized $|\text{CSD}|^2$
- $= |\text{CSD}_{XY}(f)|^2/(\text{PSD}_X(f)\text{PSD}_Y(f))$

Classical signal processing: Impulse response

Impulse = delta function $x(t) = \delta(t)$
Impulse response $h(t)$ = system response $y(t)$ to delta function input $x(t) = \delta(t)$

For linear time-invariant system, impulse response determines response to any input:

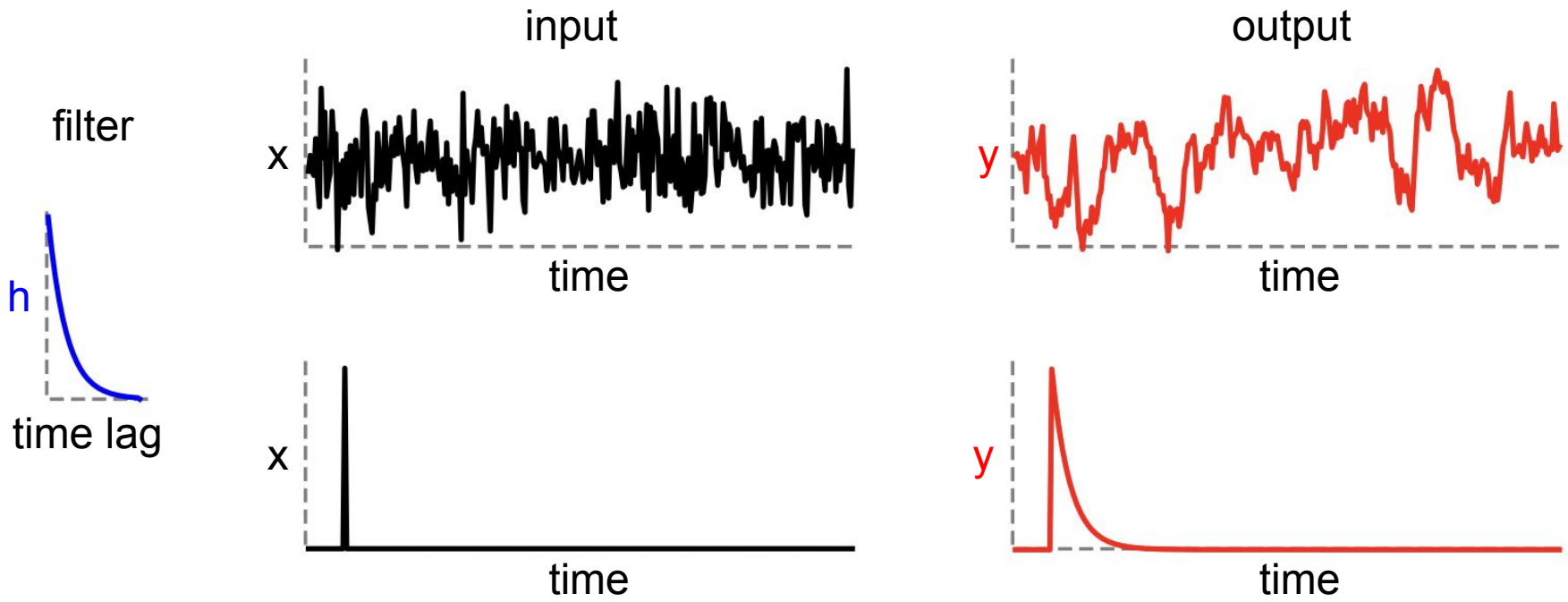
- $y(t) = h(-t) * x(t)$ where $h(t)$ is impulse response and $*$ is convolution

- $x(t) \sim P[x(t)]$, $y(t) = h * x + \text{eta} \rightarrow y(t) \sim P[x(t)|y(t)]$

Often want to estimate $h(t)$ from data with uncontrolled inputs.
- Need to be careful about multicollinearity
  - (highly correlated $x(t)$)

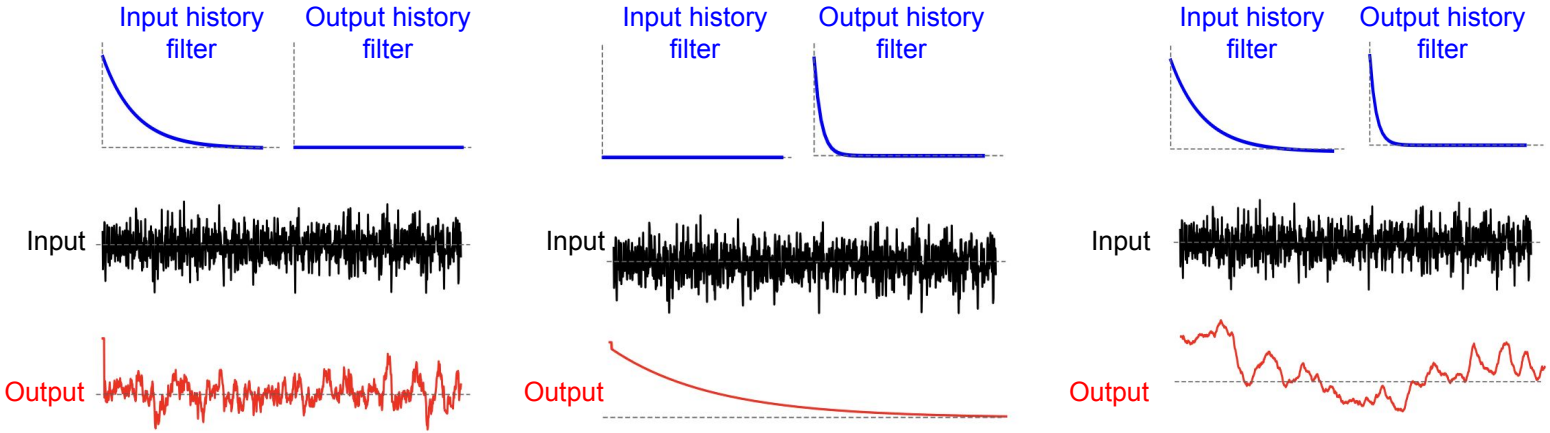
Aside: linear filters are directly related to Green's functions

Classical signal processing: Impulse response



Classical signal processing models

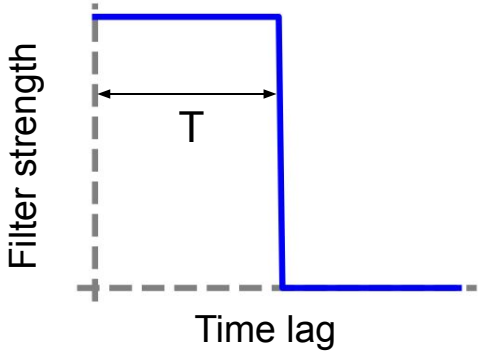
	Moving average (MA)	Autoregressive (AR)	Autoregressive moving average (ARMA)
	$Y_t = \sum_{\tau=0}^{T} w_\tau X_{t-\tau}$	$Y_t = \sum_{\tau=1}^{T} v_\tau Y_{t-\tau}$	$Y_t = \sum_{\tau=1}^{T} w_\tau X_{t-\tau} + v_\tau Y_{t-\tau}$
Output =	Weighted sum of input history	Weighted sum of output history	Weighted sum of input and output histories



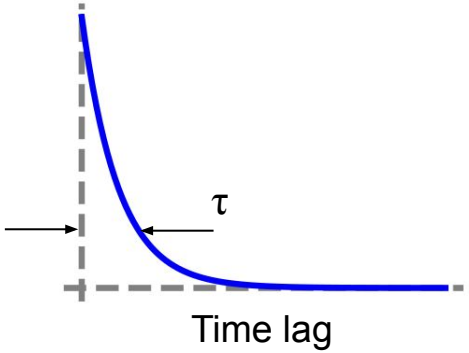
Classical signal processing models

Important filter shapes

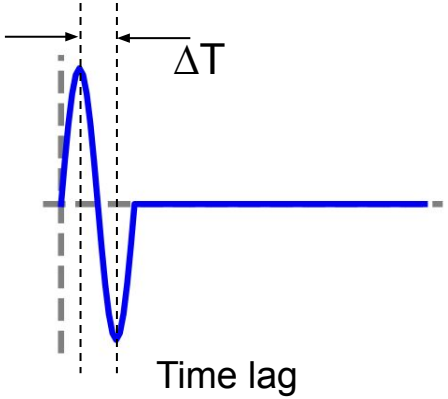
Integrator
(running average)



Leaky integrator



Differentiator



Canonical neural data analyses

Raster plots

Peristimulus time histogram

Tuning curves

Inter-spike interval analyses

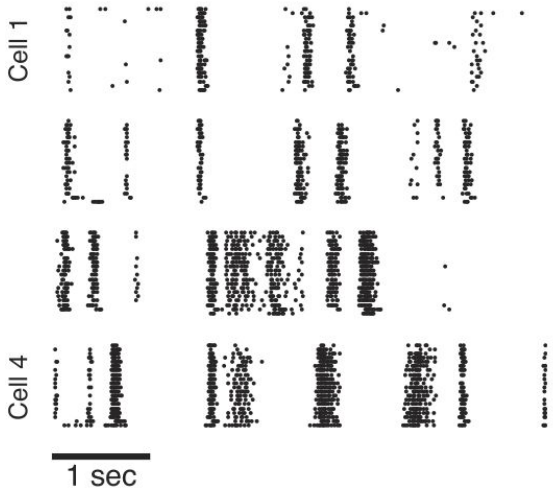
Linear-nonlinear Poisson model

Raster plot: Visualization of spikes trains across trials or neurons

Each point = spike

Each row = trial

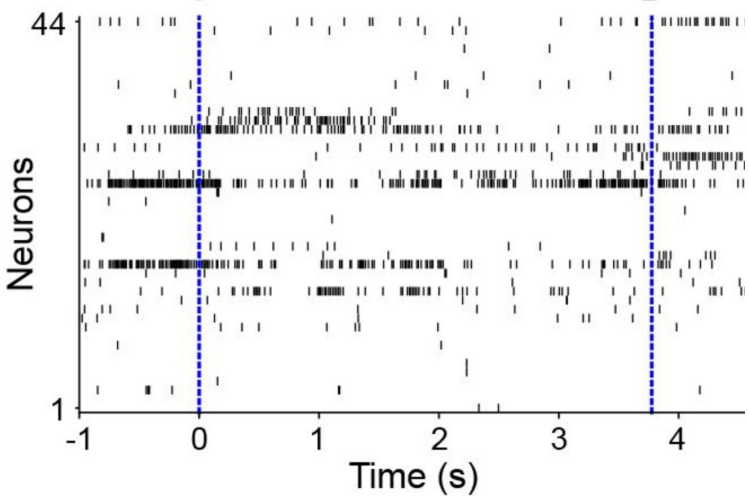
Common starting point for
single-neuron
stimulus-response analyses



Adapted from Ganmor, Segev, and Schneidman
eLife 2015, CC-BY 4.0.

Each row = neuron

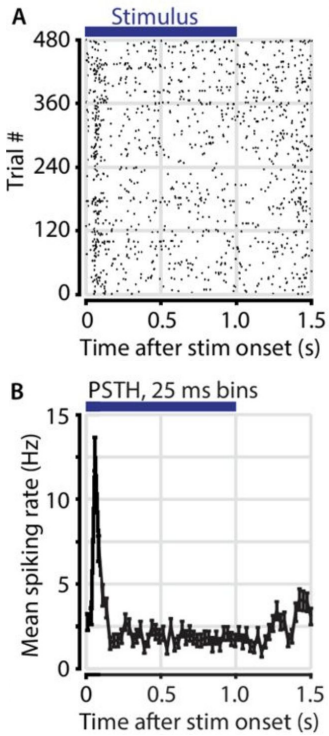
Common starting point for
analysis of population coding
and dynamics



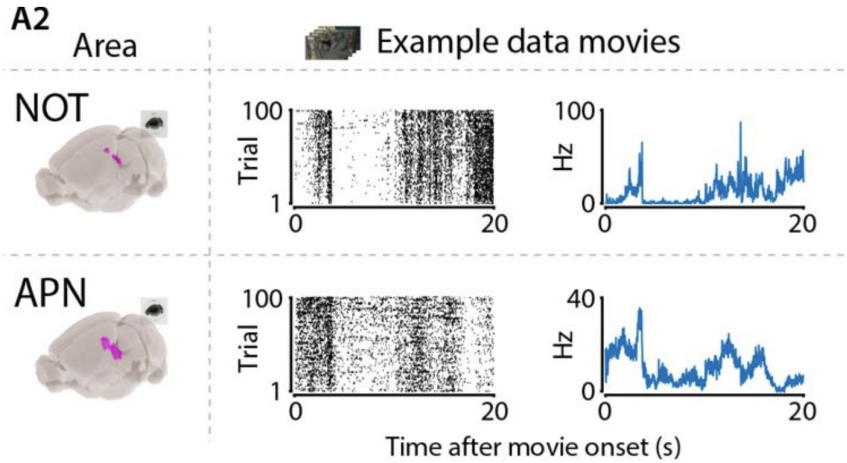
Adapted Lang et al, *eLife* 2025,
CC-BY 4.0.

Peri-stimulus time histogram

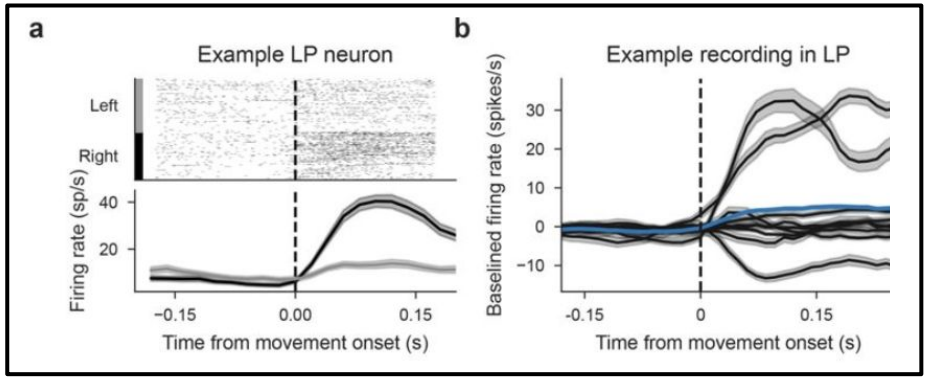
(Event-triggered average of spike trains over trials)



Adapted from Montijn et al, *eLife* 2021, CC-BY 4.0.



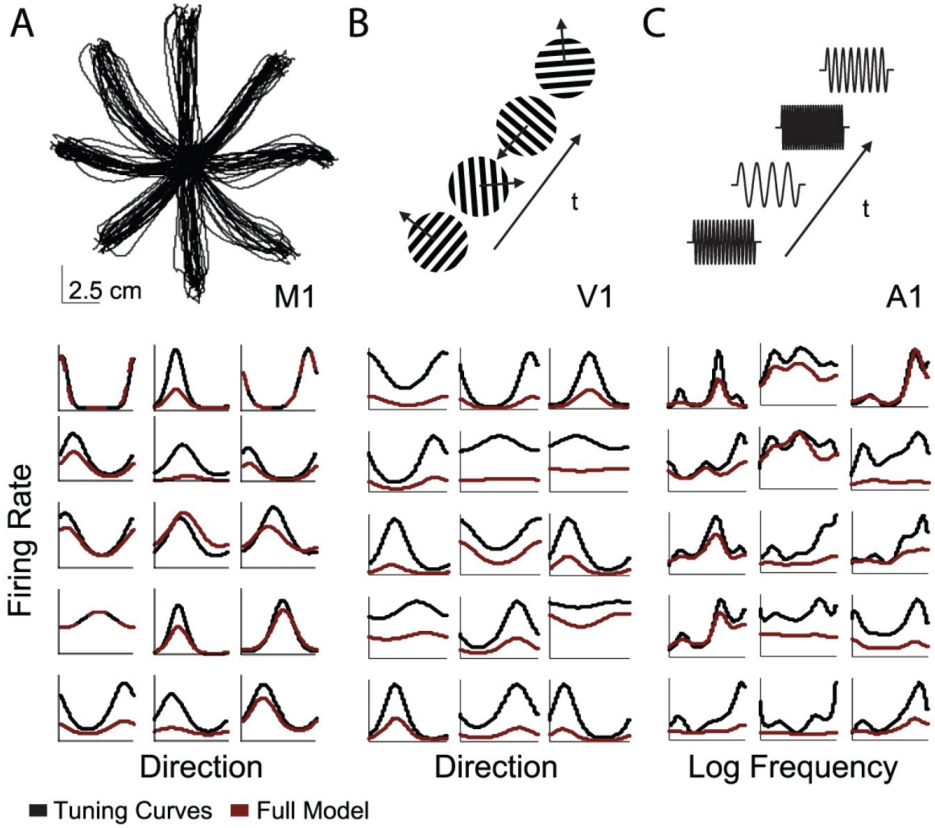
Adapted from Montijn et al, *eLife* 2021, CC-BY 4.0.



Movement-triggered average over trials

Adapted from International Brain Laboratory, et al, *eLife* 2025, CC-BY 4.0.

Tuning curves



Compute PSTH for each stimulus.

Compute time average of each PSTH to get firing rate vs stimulus.

Demonstrates what stimulus neuron “prefers”

Adapted from Stevenson et al *PLoS CB* 2012, CC-BY 4.0.

Fano factor analysis of Poisson-ness of spike counts

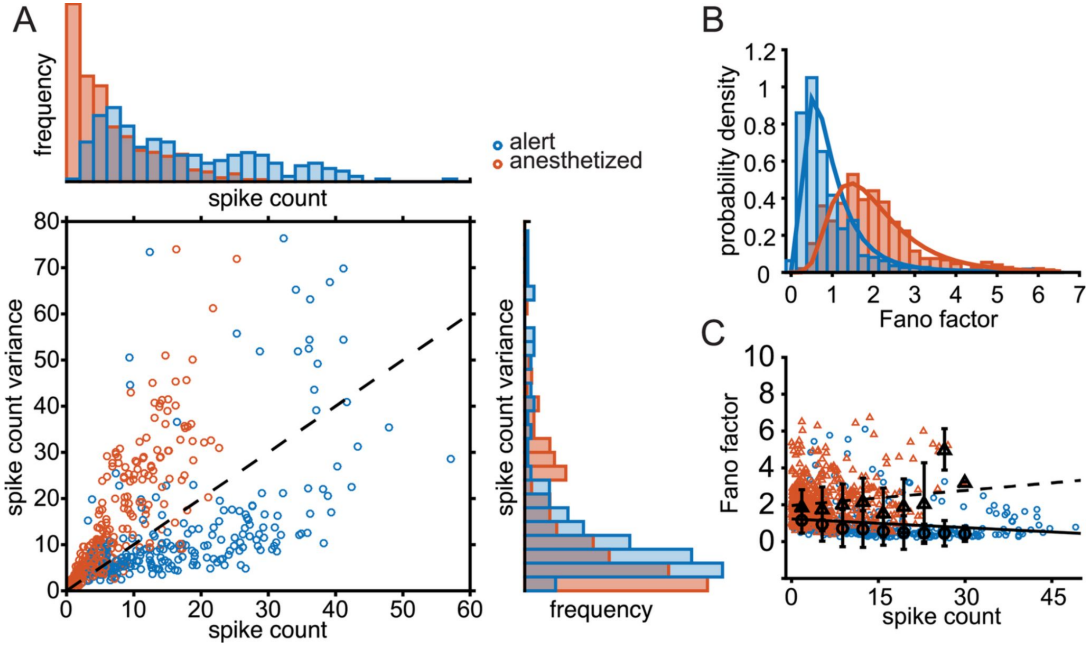
Fano factor ≡ Var/Mean

Poisson Random Variable:

Mean(k) = Var(k) = λ

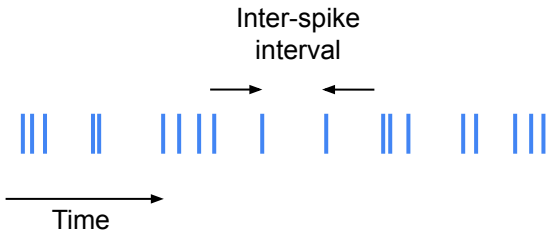
Fano factor = 1

Useful for measuring “Poisson-ness” of neuron



Adapted from Lombardo et al 2012 *PLoS CB* (CC-BY 4.0).

Inter-spike interval analysis of Poisson-ness of spike times

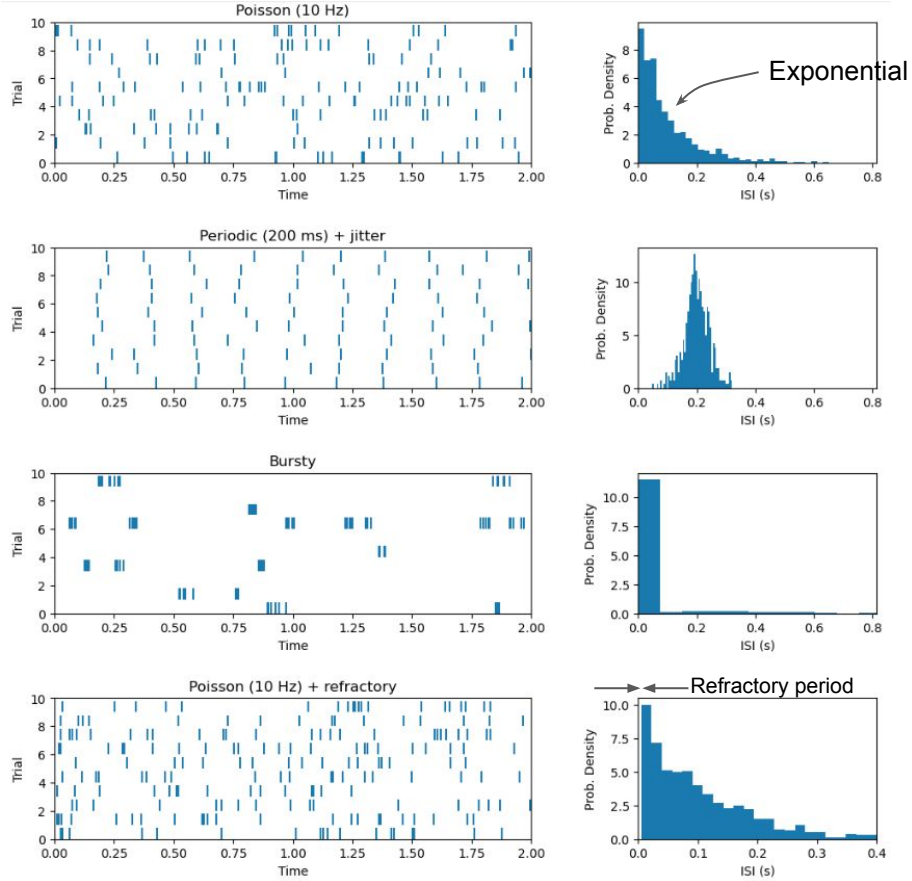


<u>Homogeneous Poisson process</u>

$\{t_k\} \sim \text{Poisson}(r)$

<u>Exponential ISI distribution</u>

$\text{ISI} \sim \text{Exp}(1/r)$



Inter-spike interval analysis of Poisson-ness of spike times

<u>Homogeneous Poisson process</u> ➡ ISI CV = 1

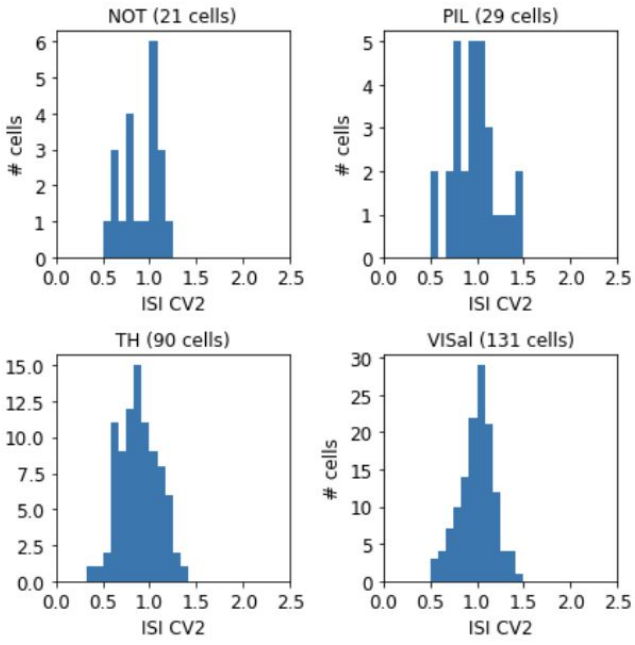
$$\text{ISI CV} = \frac{\text{SD}[\text{ISI}]}{\text{E}[\text{ISI}]}$$

<u>*Inhomogeneous* Poisson process</u> ➡ ISI $CV_2 \approx 1$

$\{t_k\} \sim \text{Poisson}[r(t)]$

$$\text{ISI CV}_2 = \sum_l \frac{2|\text{ISI}_{l+1} - \text{ISI}_l|}{\text{ISI}_{l+1} + \text{ISI}_l}$$

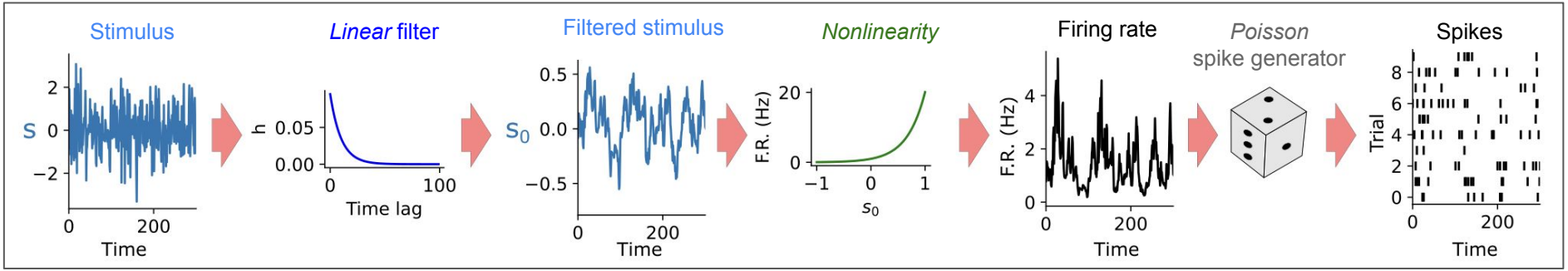
Holt et al, *J Neurophysiol* 1996.



Data from the Allen Institute for Brain Science, Visual Behavior Neuropixels dataset (CC-BY 4.0).

Linear-nonlinear-Poisson model

Model of neural response to stimulus time-series



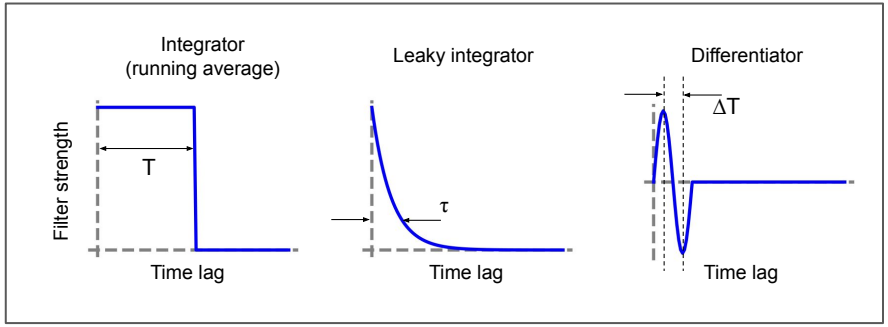
$$s_0(t) = (s * h)(t)$$

$$r(t) = g(s_0(t))$$

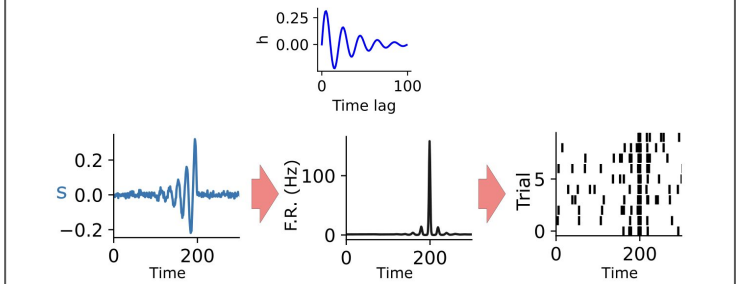
Ensures positive firing rate

$$\{t_k\} \sim \text{Poisson}[r(t)]$$

Filters that perform specific functions



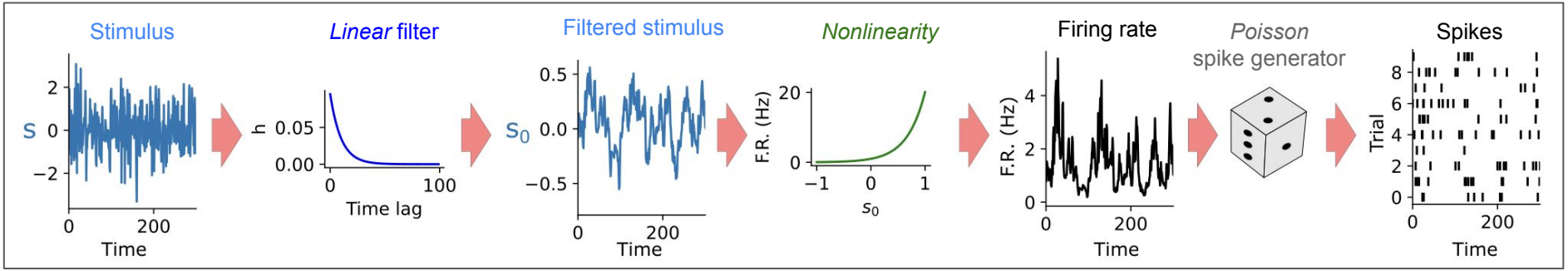
Filter as “feature detector”



Increased spike probability when stimulus “looks like” filter

Linear-nonlinear-Poisson model - Fitting to data

Model of neural response to stimulus time-series



<u>Common method</u>

Assume nonlinearity e.g. exponential.

Fit linear filter via maximum likelihood, i.e.,
Find filter weights h that maximize $P(\{t_k\}|h, s)$,
where $\{t_k\}$ and s are spikes and stimulus from data.

This can be solved with gradient descent.

* It is often helpful to parameterize or regularize h, e.g. representing h as a sum of basis functions or adding a penalty term to the objective function

* To be properly Bayesian one can also introduce a prior over h or its parameterization and then find h via maximum a posterior.

Spike-triggered average

Method to estimate linear filter and nonlinearity from data

* Do not need to assume nonlinearity ahead of time

1. Identify all spike times $\{t_k\}$
2. For each spike at t_k, save $s(t_k\text{-}T : t_k)$, i.e. last T seconds of stimulus before t_k.
3. The collection of all $\{s(t_k\text{-}T : t_k)\}$ is called the spike-triggered ensemble (STE).
4. $$\mathrm{STA} = \frac{1}{K}\sum_k s(t_k - T : t_k)$$
5. STA is a time-series $\mathrm{STA}(\tau)$ of length T
6. Estimate the LNP filter as $h(\tau) \approx \mathrm{STA}(-\tau)$
7. Compute $s_0(t) = (s*h)(t)$ using $h(\tau)$ estimate.
8. Compute $P(s_0|\mathrm{spike})$ from the STE and $P(s_0)$ from the stimulus.
9. Compute $g(s_0) \approx P(\mathrm{spike}|s_0) = P(s_0|\mathrm{spike})P(s_0)/P(\mathrm{spike})$.